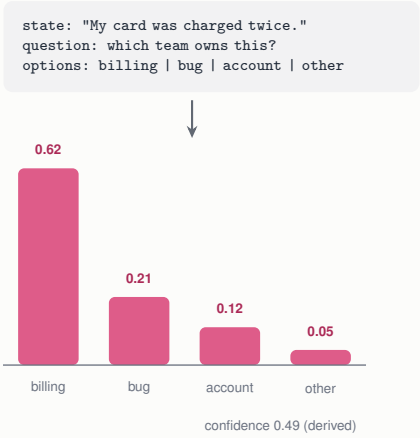
LECTURE NOTES · MODELS, 2026

Decision Models

Models that answer typed questions with calibrated probabilities instead of text: the primitives, calibration and how it is trained for, selective prediction and cascades, the 2026 landscape around TypeSafe's Jev, the failure modes, and how to put one inside an agent.



A decision model answers typed questions with probabilities

Same transformer lineage, different contract: a distribution over options you supplied, no text to parse.

a generative call

```
POST /v1/chat/completions
{ "model": "gpt-5.5",
  "messages": [{"role": "user", "content":
    "Which team owns this ticket: billing,
     bug, account or other? One word.
     Ticket: My card was charged twice."}] }
-> "Billing."    text: parse it, hope the format holds
```

a decision call (TypeSafe System One API†)

```
POST /v1/systemone
{ "model": "jev-latest",
  "state": {"ticket": "My card was charged twice."},
  "questions": { "team": { "type": "choice",
    "instructions": "Which team owns ‘ticket‘?",
    "criteria": ["billing","bug","account","other"] } } }
-> {"team": {"choice": "billing", "confidence": 0.49,
   "probabilities": {"billing": 0.62, "bug": 0.21,
            "account": 0.12, "other": 0.05}}}
```

What is different

The output is a distribution over a set you supplied: no parsing, no refusals, no format drift.

The probabilities are the contract: over many answers, “0.8” is right about 80% of the time (slides 10–12); a *confidence* summarizes the distribution (slides 6–7).

Many questions per call, independent, about 100 ms, $0.042 per million input tokens†.

What it is not

It does not write, explain, plan, call tools or reason step by step. “System One” is the vendor’s name for Kahneman’s fast, automatic judgments; the slow, deliberate work stays with reasoning models and with your code. The category is new (Jev: September 2026) and the science behind it is not: calibrated classification (slide 4).

Why now: an agent loop is full of small judgments

Most of what an agent decides is a classification, a yes/no or a rating, and a generator is an expensive way to get one.



Why a generator is the wrong tool for these

Cost and latency: a frontier model lists $5 per million input tokens and takes seconds; a decision model lists $0.042 and answers in about 100 ms[†]: 120× cheaper per input token (derived), with no output tokens to pay for.

No probability: asking a chat model "how sure are you?" gives overconfident, clustered numbers (Xiong et al. 2023); its token logprobs drift after RLHF (GPT-4 report, 2023).

Fragile format: text must be parsed; judges show position and verbosity bias (Zheng et al. 2023).

And where it is the right tool

A decision model cannot explain itself, cannot reason through a hard case, and only chooses among what you listed. The design that follows from this (slides 17–20): the cheap calibrated model decides the easy majority and *says* when a case is hard; the expensive model, or a person, gets the rest. The probability is what makes that split possible.

A short history: the parts are old, the packaging is new

Calibrated classifiers, option logprobs, reward models, judges and guards all led here; the 2026 product is the category.



What is old

Classification with a softmax trained by cross-entropy is a proper scoring rule (slide 11), so calibrated classifiers predate deep learning (Niculescu-Mizil and Caruana 2005). Reward models, guard models and rerankers are decision models in all but name: one forward pass, a number out.

What is new

A general model that takes *any* question in a few fixed shapes, with the option set in the request, trained with calibration as the objective, sold as a category ("System One", "decisions" endpoint) with a pricing and latency profile that lets it run a million times in the background. Whether it holds up is an empirical question (slides 26–27).

Four primitives: choice, score, noul, box

Every question has a fixed answer shape; the shape decides what comes back and what confidence means.

Choice

asks for one option from a set of 1–255

returns `choice`, a probability per option, `confidence`

e.g. *Which team owns this ticket? billing — bug — account — other*

Score

asks for a position on 2–10 ordered levels

returns `score` (expected level), a probability per level, `confidence`

e.g. *How severe is the issue? 0 cosmetic, 1 degraded, 2 blocking*

Noul

asks for a yes/no judgment

returns `probability` that the answer is yes; no separate confidence

e.g. *Does the quoted passage support the cited claim?*

Box

asks for where something is in an image

returns `top`, `bottom`, `left`, `right` as fractions of the image; `confidence`

e.g. *the coffee mug in 'photo' (Jev 1.14, opt-in header)*

Choosing a shape

One known category should win → **Choice**. Is a property present, with a probability → **Noul**. An ordered rubric, severity, relevance, quality → **Score**. Where is it → **Box**. A broad judgment is several of these; ask them all in one call (slide 9).

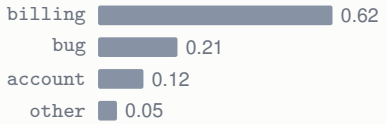
Status, October 2026†

Choice, Score and Noul are the production primitives on Jev 1.13 and 1.14. Images (PNG/JPEG, 280–1568 px per side, 16 MiB) go in state, instructions or criteria behind the header `TypeSafe-Danger-Zone-Images: true`; Box needs Jev 1.14. The vendor's words for images and Box: "aren't yet up to our standards to call ready for production."

Choice: a distribution over your options, plus one number

The answer is the argmax; the probabilities are the real output; confidence is how far the top sits above an even split.

`probabilities` for the ticket on slide 2 (example)



$$\text{confidence} = \frac{p_{\max} - \frac{1}{n}}{1 - \frac{1}{n}} = \frac{0.62 - 0.25}{0.75} = 0.49$$

derived from the docs' formula with $n = 4$

Reading it. Confidence is 0 at an even split and 1 when all mass is on one option. Only $p_{\max}$ counts: $(0.6, 0.3, 0.1)$ and $(0.6, 0.2, 0.2)$ both give 0.4; the docs offer the top probability or the top-to-second ratio as alternatives, and entropy is a third. Pick one and calibrate your thresholds against it (slide 18).

Designing the option set

Make options exhaustive. The model must put its mass somewhere. If "none of these" can happen, list it; the vendor's self-consistency recipe adds an explicit *uncertain* outcome.

Describe each option with `what`, `not_for` and `examples`; confusable neighbours are where errors live.

Up to 255 options, but a long flat list is a weak question; hierarchies are better searched level by level with a beam over the path probabilities (slide 20).

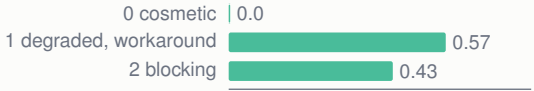
Order matters more than it should: first-option bias is a documented rough edge (slide 24). Shuffle, or ask twice in two orders and average.

A two-option Choice is a Noul with a confidence attached; the formulas agree ($|2p - 1|$).

Score: ordered levels, an expected level, and a spread

The score is the probability-weighted level, so it can sit between levels; confidence is how tight the mass is.

`bug_severity`, three levels (vendor example†)



$$\text{score} = \sum_i i\, p_i = 0.57 + 2 \cdot 0.43 = 1.43$$

$$\text{confidence} = \max\Big(0,\ 1 - \frac{\sum_i p_i\,|i - m|}{\mathrm{MAD}_{\text{unif}}}\Big)$$

$$= 1 - 0.43/0.667 = 0.35$$

m = most likely level; $\mathrm{MAD}_{\text{unif}} = \frac{1}{n}\sum_i |i - \frac{n-1}{2}|$ is the spread of an even distribution ($\frac{2}{3}$ for $n = 3$); derived from the docs' formulas

Why not the Choice formula? Levels have distances: $(0, 0.5, 0.5)$ gives 0.25 but $(0.5, 0, 0.5)$ gives 0. Mass two levels apart is a real disagreement; mass on neighbours is not.

Using scores

2 to 10 levels, numbered from 0, each with a description and ideally `signals` or examples; the response echoes a `legend`.

A score of 1.43 is not a label. It is an expectation. Round it only when a label is needed; keep it when summing or ranking.

The same idea elsewhere: G-Eval (Liu et al. 2023) made LLM judges score with the probability-weighted sum over rating tokens instead of the argmax, for finer, less tied scores. Ordinal regression is the classical name.

Composite scoring: ask several atomic Score questions (clarity, accuracy, tone) and combine with weights in code; the weights are yours to audit, not the model's.

Thresholds on the score, not on the mode: the guardrail recipe turns a review into a block at severity ≥ 2.0†.

Noul and Box: a single probability, and four edges

Noul is the workhorse for verification and detection; Box is the first step outside text, and still a preview.

Noul: `probability` that the answer is yes



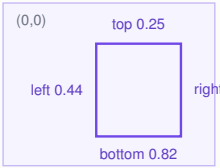
Reading a Noul

No separate confidence: "the number is the answer and the certainty in one"†. A value near 0.5 is the model saying it cannot tell. If you want a 0–1 certainty, use $|2p - 1|$, which is exactly the Choice formula for two options.

Define both sides. Criteria should say what counts as yes *and* what counts as no, with examples; otherwise the model decides where the boundary is.

Two thresholds, not one: the guardrail recipe reviews at $p \geq 0.35$ and acts at $p \geq 0.70$ (strict) or 0.85 (permissive)†; the band between is the human's.

Box: edges as fractions of width and height



the docs' mug example†: top 0.25, bottom 0.82, left 0.44, right 0.84; confidence 0.92; origin top-left.

Using Box (preview†)

Always returns a box, even when the target is absent: pair it with a Noul asking whether the thing is there. One target per question. Targets a few dozen pixels wide in a large image can be missed: crop first. Needs `jev-1.14` and the images header; format and limits may change between releases.

State and questions: facts go in, judgments are asked

One request evaluates one state against many independent questions; batching costs nothing in accuracy.

```
"state": {
 "ticket": {"subject": "double charge", "msgs": [..]},
 "order": {"id": "A-104", "charges": [49, 49]},
 "refund_policy": "Duplicate charges are refunded." },
"questions": {
 "team": {"type": "choice", "instructions": "...",
       "criteria": [..]},
 "refund_ok": {"type": "noul", "instructions":
  "Is a refund due for ‘order‘ under ‘refund_policy‘?"}}
```

12.2×
cheaper: 13 questions in one call, not 13 calls†

10.0×
faster: 0.27 s against 2.71 s summed†

vendor cookbook, `jev-1.12`: 8 Noul, 2 Choice, 3 Score questions on a 53,777-character GDPR article, 5 runs; $0.000497 vs. $0.00609; answers identical on 11 of 13

Rules from the docs†

State is a string, an object or an array; prefer an object so questions can point at parts with backticked paths. Include only what the questions need: irrelevant state hurts (slide 24).

Questions are short, atomic, explicit: “broad questions hide several judgments behind one answer.” Deterministic rules (days overdue, amounts) stay in code. All questions see the same state, independently.

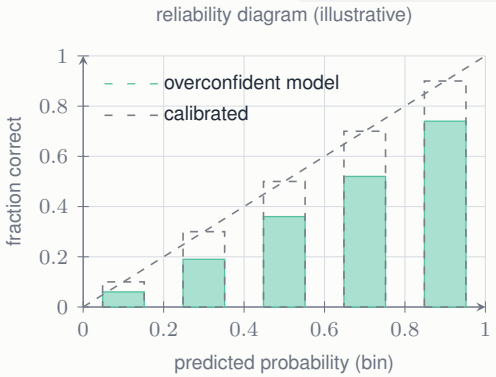
Request rules†

64k tokens per request, 32k per question (Jev 1.13/1.14); body $\leq$ 32 MiB, gzip or zstd; nesting $\leq$ 64; finite numbers only. `TypeSafe-Danger-Zone-Long-Context: true` raises both limits to 262k at the same price, “slower and less accurate.” Output tokens are free, so a request costs its input: a 10k-token state with 20 questions is about $0.0004 (derived).

Calibration: the probability has to mean something

Of all the answers given with probability p, about a fraction p should be right: a population property.

after Guo et al. 2017; DeGroot & Fienberg 1983



Three numbers

Expected calibration error: bin the answers by predicted probability and average the gap, $\mathrm{ECE} = \sum_b \frac{n_b}{N} |\mathrm{acc}_b - \mathrm{conf}_b|$. Cheap, standard, sensitive to the binning (slide 26).

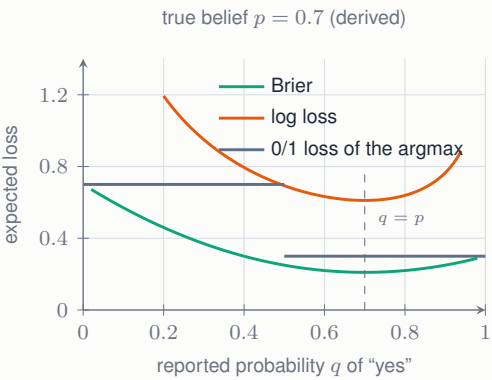
Brier score: $\frac{1}{N} \sum_i \sum_k (p_{ik} - \mathbb{1}[y_i{=}k])^2$, squared error of the probability vector. Murphy: Brier $=$ reliability $-$ resolution $+$ uncertainty, so a model can be calibrated and useless (always the base rate) or sharp and wrong.

Log loss: $-\frac{1}{N} \sum_i \log p_i(y_i)$, the cross-entropy a classifier is trained on; unbounded near $p = 0$.

What a decision model promises: its `probabilities` sit on the diagonal for *your* questions. The claim is made on the vendor's data; slides 26–27 ask how far it transfers.

Proper scoring rules make honesty the best policy

Score the probability, not the guess: a strictly proper rule is optimized only by reporting the true belief.



Why this matters for decision models

Brier $p(1-q)^2 + (1-p)q^2$ and log loss $-p \ln q - (1-p)\ln(1-q)$ are strictly proper (Brier 1950; Gneiting and Raftery 2007): the curve bottoms out exactly at $q = p$. Train or reward a model with one of them and the calibrated answer is the optimal answer; a softmax classifier trained by cross-entropy already does this.

Accuracy is not proper. Any $q > 0.5$ scores the same, so a model graded on hits has no reason to report 0.7 rather than 0.99, and every reason to guess rather than abstain. Kalai et al. (2025) make this the explanation for hallucination: benchmarks give 0 for "I don't know" and sometimes 1 for a bluff.

Preference rewards are not proper either. Reward models prefer confident-sounding text (Leng et al. 2024), which is why RLHF erodes calibration (slide 12).

Do language models know what they know? Until post-training

Base models are calibrated in the right format; preference tuning breaks it; asking in words helps but overshoots.

Deep nets drifted out of calibration

Guo et al. (2017): a 2016 ResNet is far more overconfident than a 1998 LeNet; depth, width and weight decay all matter; one temperature fitted on held-out NLL fixes most of it (slide 16). Minderer et al. (2021): architecture is a major determinant.

Post-training breaks it

GPT-4 report (2023, Fig. 8): ECE on an MMLU subset 0.007 before, 0.074 after PPO: "the post-training hurts calibration significantly." Xiao et al. (2025): preference tuning collapses the probabilities; calibration-aware fine-tuning takes ECE from 14–20% back to 2.4–6.5%.

Base LMs are calibrated when the format fits

Kadavath et al. (2022): large models are well calibrated on multiple-choice and true/false "in the right format" and can predict $P(\text{True})$ of their own samples; a format shift degrades it. Zhao et al. (2021): a content-free input exposes a label prior; correcting it adds up to 30 points.

Words help, but overshoot

Tian et al. (2023): for RLHF models a verbalized "80% sure" beats the token probability (gpt-3.5, TriviaQA: ECE 0.140 $\rightarrow$ 0.050). Xiong et al. (2023): still overconfident and clustered. Kapoor et al. (2024): about 1,000 graded examples plus LoRA is what generalizes.

The lesson a decision model is built on: calibration is cheap to have and easy to lose; it survives when the model is trained and scored in the exact format it answers in (and none of nine recent model reports even report it: Sanz-Guerrero and von der Wense 2026).

Four ways to get a probability out of a language model

Read the logits, ask in words, sample and count, or train a head. A decision model is the fourth, sold as a service.

Method	Needs	For it	Against it
Option logprobs (MMLU style: "Answer: A/B/C/D")	logits, or a `logprobs` API; one forward pass	cheapest; the classical classifier readout; fixable with a temperature or a label-prior correction (Zhao et al. 2021)	token bias on the letters (Zheng et al. 2023); order sensitivity; first-token and text answers disagree at rates reaching over 60% for instruction-tuned models (Wang et al. 2024)
Verbalized ("how sure are you?")	any chat API	works on closed models; readable; best option for RLHF models (Tian et al. 2023)	overconfident and clustered (Xiong et al. 2023); prompt-sensitive; costs output tokens
Sampling consistency (agreement over k samples)	any API, $k\times$ the cost	works for free-form answers; a good difficulty signal for cascades (Yue et al. 2023)	measures stability, not truth; k calls per decision
Trained head or decision model	labels, or a vendor	trained on a proper scoring rule in the answer format; one pass; batched questions; no text	fixed question shapes; no reasoning; calibration certified on someone else's data (slide 27)

Why the fourth wins on cost: verbalizing and sampling pay for generated tokens; a decision head emits a few numbers per question, which is what makes "output tokens are free" plausible. Quality: slide 27.

What is known about how a decision model is trained

The vendor gives a name and a goal; the published recipes that match the goal all reward a proper scoring rule.

What TypeSafe says[†]

RLCD, "reinforcement learning for calibrated decisions," trains the model "to return decisions and calibrated probabilities instead of generated text." Calibrated: of the answers given 0.2, about 20% come true; of those given 0.8, about 80%; rates over groups, never one answer.

Nothing else is public: no paper, weights, architecture or parameter count; the name collides with RLCD of Yang et al. (2023), contrastive distillation, unrelated.

Treat it as a claim, as Rafe and Das (2026) do, and test the probabilities on your own labels (slide 27).

The nearest published recipes

Lin, Hilton, Evans (2022): fine-tune GPT-3 to state a verbal probability; calibrated, and it survives distribution shift.

Rewarding Doubt (Bani-Harouni et al. 2025): an RL reward that is a log scoring rule on the stated confidence; the optimal policy is exactly calibrated.

RLCR (Damani et al. 2025): reward = correctness + Brier score of the stated confidence; "ordinary RL hurts calibration, RLCR improves it," at no cost in accuracy.

Variations: SaySelf, LACIE, Leng et al. (all 2024).

What this implies (inference, not disclosure). The output is a distribution over supplied options, so the natural reward is a proper scoring rule against a label or an outcome; RL rather than cross-entropy buys rewards from outcomes and one policy over every question shape.

RLHF, RLVR, RLCD: three objectives, three kinds of model

What the signal is, what the policy emits, what gets rewarded, and what each one is known to get wrong.

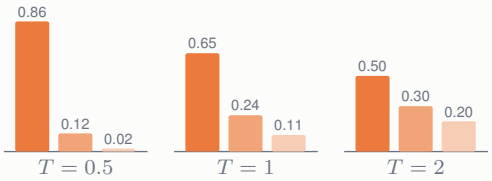
	Signal	The policy emits	Rewarded for	Known failure
RLHF (InstructGPT 2022)	human or AI preferences, via a reward model	free text	what raters prefer	sycophancy, confident-sounding errors; probabilities collapse onto the preferred answer (slide 12)
RLVR (DeepSeek-R1, Tülu 3)	a verifiable outcome: unit test, final answer	a chain of thought, then an answer	the right final answer	long reasoning on easy cases; overconfident when wrong (Mei et al. 2025); reasoning tuning hurts abstention (AbstentionBench 2025)
RLCD (TypeSafe 2026†)	labelled or verifiable decisions scored by a proper rule (inferred)	a distribution over the supplied options; no text	a calibrated probability, not only the right argmax	undisclosed; the vendor's own jaggedness list (slide 24); flips under injected context (slide 25)
RLCR (Damani et al. 2025)	outcome + Brier on the stated confidence	reasoning, answer, confidence	correctness and calibration	the academic hybrid; still generates, so still slow

Inference follows the objective: RLHF and RLVR pay per generated token and take seconds; a decision model is one forward pass, about 100 ms†. Complements, not rivals: the reasoning model takes the hard cases the decision model flags.

Calibration after the fact: temperature, Platt, isotonic

One scalar fitted on a few hundred labelled examples repairs most miscalibration without touching the argmax.

softmax of logits $z = (2.0,\ 1.0,\ 0.2)$ at three temperatures (derived)



$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)},\ T \text{ fitted by minimizing held-out NLL (Guo et al. 2017)}$$

Platt (1999): $\sigma(az + b)$. Isotonic regression (Zadrozny and Elkan 2002): a monotone step function. Dirichlet calibration (Kull et al. 2019): a linear map on log-probabilities for many classes. $T > 1$ flattens, $T < 1$ sharpens; the argmax never moves.

What it does and does not do

Needs labels from your distribution: a few hundred to a few thousand graded answers; the fit is one or two numbers, so it rarely overfits.

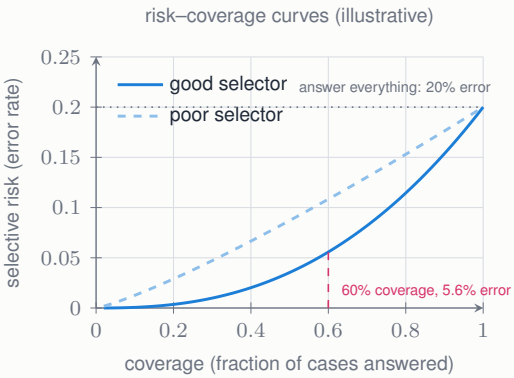
Leaves ranking and accuracy alone: it moves the reliability diagram onto the diagonal; it cannot make a bad selector good (slide 17).

Breaks under shift: calibrated on one domain, off on another (Ovadia et al. 2019); Kumar et al. (2019): scaling looks better calibrated than it is; bin after scaling to verify.

It applies to decision models too. Rafe and Das (2026): raw Jev probabilities overstated prevalence; recalibrating on 2,416 human labels cut the error 3.3×. Dibia (2026)[†]: a 7B logprob readout went from ECE 0.10 to 0.03 with one temperature. Whoever trained the model, calibrate on your own labels.

Selective prediction: answer when sure, hand off the rest

Sort decisions by confidence and stop answering below a threshold; error against coverage is the trade-off.



The formalism

A selective classifier is a pair (f, g): f predicts, $g(x) \in \{0, 1\}$ says whether to answer (El-Yaniv, Wiener 2010). Coverage $\phi = \mathbb{E}[g]$; selective risk $R = \mathbb{E}[\ell(f(x), y)\, g(x)]/\phi$. Take $g = \mathbb{1}[\mathrm{conf}(x) \geq \tau]$ and sweep τ; the area under the curve (AURC, Geifman et al. 2019) scores it.

Ranking, not calibration, is what matters here: confidence must order right and wrong answers (AUROC; Hendrycks and Gimpel 2016). Guo et al. (2026): Jev ranks well (median AUROC 0.886 over 31 yes/no benchmarks) while its per-benchmark ECE (median 0.168, against 0.074 expected under perfect calibration) is poor; pooled, 0.047: calibrated on average, off per task.

Deferral: if rejected cases go to a person or a larger model, their error counts too (Mozannar and Sontag 2020).

Thresholds are a cost decision, not a model setting

A calibrated probability turns “how sure must we be?” into arithmetic on the cost of each kind of mistake.

act when $p \geq p^* = \dfrac{c_{\mathrm{FP}}}{c_{\mathrm{FP}} + c_{\mathrm{FN}}}$

Elkan (2001), with correct decisions costing 0; costs in any unit you like; illustrative rows

Decision	c_{FP}	c_{FN}	p^*
flag a message for a safety reviewer	1	50	0.02
auto-approve a $49 refund	49	5	0.91
route a ticket to the billing queue	2	2	0.50
trigger the expensive re-extraction	1	20	0.05

Read down the last column: one model, one probability, four thresholds, all in code and auditable. Calibration is what makes a threshold transferable; miscalibrated, it is a tuned number, re-tuned per question and per model version.

The vendor’s bands[†]

Three bands: act automatically at high confidence, proceed with caution in the middle, do not act at the bottom. Their voice-banking example: confidence < 0.6 on any intent $\rightarrow$ a support agent; `check_balance` at $\geq 0.6 \rightarrow$ show it; `approve_transfer` at $> 0.85 \rightarrow$ approve, in $[0.6, 0.85] \rightarrow$ ask the user to confirm (the confidence page gives the same example with 0.9 and a 0.5 floor). The guardrail recipe reviews at 0.35 and blocks at 0.70. “Start with conservative thresholds, test with your own data, and adjust as you observe results.”

How to test: plot confidence against accuracy on a labelled sample of *your* questions, read off where the error crosses what each action tolerates, and re-check whenever the alias moves (`jev-latest` does).

Conformal prediction: sets with a coverage guarantee

From any probability vector to a set that contains the right answer $1 - \alpha$ of the time, with no model assumptions.

Split conformal in four lines (Angelopoulos and Bates 2021)

1. On n labelled calibration questions compute the nonconformity score $s_i = 1 - \hat{p}(y_i \mid x_i)$.
2. Let $\hat{q}$ be the $\lceil (n+1)(1-\alpha) \rceil / n$ quantile of the s_i.
3. For a new question return $C(x) = \{\, y : \hat{p}(y \mid x) \geq 1 - \hat{q} \,\}$.
4. Then $\Pr[\, y \in C(x) \,] \geq 1 - \alpha$, marginally, whenever the new question is exchangeable with the calibration set (Vovk et al. 2005).

The size of the set is the uncertainty: $|C| = 1$ act; $|C| > 1$ ask; $|C| = 0$ the model has never seen anything like it.

three tickets at $\hat{q} = 0.88$: keep options with $p \geq 0.12$ (illustrative)

billing .91	bug .03	account .04	other .02	$\lvert C \rvert = 1$: route it
billing .62	bug .21	account .12	other .05	$\lvert C \rvert = 3$: ask a person
billing .40	bug .38	account .15	other .07	$\lvert C \rvert = 3$: ask a person

In the literature, and in practice

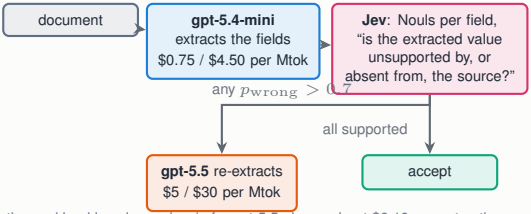
Kumar et al. (2023): conformal sets over LLM option probabilities; set size tracks accuracy. Ren et al. (2023, KnowNo): a robot plans with multiple-choice next steps and asks a human exactly when $|C| > 1$. Mohri and Hashimoto (2024): drop sub-claims until the retained text meets a correctness target.

With a decision model this is a ten-line wrapper: the Choice probabilities are the scores. The guarantee covers the set, not the argmax, and only holds while traffic resembles the calibration set.

Cascades and fan-out: the cheap model decides who pays

A small calibrated model in front of a large one, firing it only when flagged; and many questions asked in one pass.

the structured-extraction cascade from the vendor cookbook†



the cookbook's only number is for gpt-5.5 alone: about $0.10 per extraction at “≈0.81 quality” on an undefined vendor score; the cascade is shown as a curve, “most of the top model's quality at a fraction of its cost.”

Speculative fan-out†

Ask every question you might need about a state in one call, the speculative ones included, and let code decide afterwards which answers matter: one round trip instead of a chain (slide 9).

Cascades and routers in the literature

FrugalGPT (Chen et al. 2023): a cascade of LLMs with a learned scorer matches GPT-4 at up to 98% lower cost, or adds 4% accuracy at the same cost.

Jitkrittum et al. (2023): deferring on the small model's confidence is optimal only if the large model's error is roughly constant across inputs; otherwise learn the deferral rule from both models' errors.

RouteLLM (Ong et al. 2024): preference-trained routers cut cost more than 2× at matched quality. **Yue et al. (2023):** answer consistency as the difficulty signal.

A decision model as the gate: Wu and Lim (2026) report 95% task success with 72.7% fewer strong-model calls in an agent, and little gain where a cheap cascade already routes well.

Jev: the product that named the category

Five weeks old on the day of this lecture; a price, a latency and a request volume that explain the attention.

$0.042
per million input tokens†

78–127 ms
median per task, 1.14 / 1.13

64k / 32k
tokens per request / question

956M
requests in one week, Jev 1.13

output free; latency from the Artificial Analysis Decision Index; 262k behind a header†; OpenRouter decisions ranking, week to Oct 20

The company and its claims†

TypeSafe AI, San Francisco, founded 2024; CEO Diogo Almeida, ex-OpenAI, an InstructGPT co-author. Jev 1.13 shipped September 15, 2026 with a $40M seed; 1.14 on October 14; an $870M round at $7.5B on October 9. Vendor statements: “hundreds of outputs in parallel from a single prompt” (press release); training data all synthetic (the CEO, in interview); no paper. Its own workflow evaluation puts Jev at 67.8% against GPT-5.6 Sol’s 74.1% at 1/200 of the cost (derived), with LLM outputs as the reference labels.

Measured by others

Latency: OpenRouter’s Banking77 run at 8 concurrent requests: p50 / p95 / p99 of 175 / 270 / 353 ms against 2,266 / 3,004 / 3,835 ms for Claude Opus 5; Anaconda: median 88 ms, but one request in twenty about 7 s.

Cost: $0.11 per 1,000 Banking77 classifications, against $2.42 for Opus 5.

Availability: the API “briefly lost the ability to serve users” in launch week (TechCrunch); 529 “Overloaded” is a documented status; `jev-latest` moves after each release: pin versions.

The landscape, October 2026: one category, many insides

OpenRouter's guide lists 22 decision models from 13 makers (Oct 16); the open ones show how different the insides are.

Maker / model	What is disclosed about the mechanism	$/M in	ctx	weights	listed
TypeSafe Jev 1.13, 1.14	undisclosed: "parallel sampler," RLCD; choice/score/noul, box on 1.14	0.042	64k	closed	Sep 18, Oct 14
OpenAI gpt-6-luna Decisions	`/v1/decisions`, public beta; predicate/choice/score (refusal as an answer type); "about 10× faster than the Responses API"; no calibration claim	0.10	1.1M	closed	Oct 6
Microsoft-Decision-1	post-trained Qwen3.5-9B, single-pass scoring; vendor reports zero flips on option reversal or shuffle	0.042	32k	none found	Oct 9
Cloudflare Clef, flash, omni	frozen Qwen3.8-27B / Qwen3.5-9B + joint schema head, rank-256 LoRA; loss = CE + Brier + ordinal partial credit; omni on Qwen3-Omni-30B-A3B with audio and video	0.042 / 0.021 / 0.15	66k	Apache-2.0	Oct 1, 9
Perplexity pplx-decider v1.2, flash	Qwen3.8-27B + a 255×5120 readout head at a marker token per question; non-causal attention; $T = 1.38$; dev ECE 0.014	0.02 / 0.01	262k	Apache-2.0	Oct 16
Liquid d1, d1-ultra	"one forward pass, no tokens generated"; built on a larger open-weight model; billed per question	0.02 / 0.042	66k	closed	Oct 1, 15
Together Tev1-4B	SFT of Qwen3.5-4B that *generates* one option letter; choice only, 2–24 options; recipe published	0.042	33k	open	Sep 30

Also listed: Nace.AI Drex (v1.5 open), Inception Mercury Decide, Upstage Solar Decide, Respan Span, ScaleDown; the community Kev-4B is on slide 23. Weekly requests: Jev 1.13 956M, 1.14 257M, then Drex 38M, Decider 37M and 29M, Clef 25M, GPT-6 Luna 24M, Mercury 22M, Drex v2 22M, Clef Flash 20M: Jev is about 85% of the top ten (derived). Sources: OpenRouter pages and rankings to Oct 20, vendor blogs, model cards†.

Open weights: four ways to build one, all on Hugging Face

Read the letters, generate a letter, bolt a head onto a frozen LLM, or fine-tune an encoder; llama.cpp serves them all.

(a) Logprob readout from a stock LLM

OpenJev 27B: letter the options, ask for one token, map the letter logits to probabilities with a fixed calibration; shuffling flips 2.3%†. "LLM-as-Jev" (Li and Wagle 2026): sum the option-suffix logprobs after "Best answer: [": a 4B model, training-free, 81.4% on JevBench at ECE 0.057. AnyJev: rotate the options and take the log-mean to tame position bias†

(b) A decision head on a frozen LLM

Kev-4B: a pointer head scores each option's closing token against a `<decide>` token; rank-16 LoRA, 33.8M trainable parameters. Against Jev on its card: MMLU-Pro 0.565 vs 0.840, Brier 0.243 vs 0.211†. Perplexity's decider and Cloudflare's Clef are the same idea at 27B, with published losses (slide 22).

(c) An encoder classifier

Laya, 421M: ModernBERT-large with a two-layer head scoring each option at its own `[MASK]`; Apache-2.0, 54k downloads a month; Banking77 0.425 against Jev's 0.870 (on slightly different label sets); 20th of 23 on S1MB†. Julia-1, 144M (mmBERT-small). Raschka's phrase for the whole category: "the plug-and-play version of dedicated classifiers."

(d) Generate a letter, and serving

Together's Tev1-4B fine-tunes a 4B model to emit one option letter: a generator in a decision model's clothes. **llama.cpp** added `/v1/systemone` on Oct 2: per question on an RTX PRO 6000, Julia-1 3 ms, Laya 5 ms, Kev-4B 12 ms, OpenJev 27B 43 ms†; SGLang and vLLM serve Clef on the same Jev-compatible endpoint.

Jaggedness: where a decision model is rough

The vendor publishes its own list; independent tests confirm it and add wording sensitivity.

The vendor's own list for Jev 1.13[†]

Literal reading: criteria are taken at their word. **Counting, arithmetic, date comparison:** unreliable; do them in code and pass the result in state. **Double negatives and indirection** hurt. **Irrelevant context** ("context rot") measurably lowers accuracy: send only what the questions need. **Adversarial content is treated as data:** instructions inside the state can steer the answer. **Contradictory criteria** give arbitrary answers. **Option order:** 1.13 "leans toward the option that comes first." **No text:** it cannot say why.

What others measured

Order: Anaconda, 80 cases reversed: 79 same answers, but probabilities moved by up to 0.20 and identical requests varied by up to 0.04; Archestra: about 4 of 100 agent decisions changed on reversal. Open readouts are worse: Archestra's 0.6B model always chose A; raw Qwen3-8B flipped 23% (AnyJev).

Wording: TypedBench (arXiv 2610.11392), 500 tickets: median 0.874, but 0.708–0.946 across paraphrases of the same question, "systematically underconfident."

Literalness: "5-minute walk" → "5-minute drive" flipped 0 of 20 to 20 of 20 answers in a 7,805-request study.

The literature predicted most of this. Position and token bias in multiple choice (Zheng et al. 2023; Pezeshkpour and Hruschka 2023: 13–75% swings from reordering); judges flipped by swapping positions (Wang et al. 2023). Mitigations that transfer: shuffle or rotate and average, correct the label prior (PriDe), list "none of these," and keep the state short.

What a calibrated probability cannot tell you

Calibration is about a population of questions; the state may be written by an adversary; the label is not the outcome.

It is relative to a distribution

Ask Jev which side a fair coin landed on and it answers 0.92 for one side; on 400 die rolls, about 83% on one face (community tests†). Nothing in the request carries the base rate, so no answer can be calibrated for it. “Calibrated” means over the vendor’s distribution; yours differs (slide 27: ECE 0.02 to 0.25).

The state is attacker-controlled

JevOut (Xu et al. 2026): fluent context additions flipped 61–73% of initially correct decisions across four decision systems and seven datasets. JevAdvBench (Hu et al. 2026), 9,744 single-edit attacks: an appended unverified opinion flips 12.1% of decisions and drags 38% of confident answers under a 0.8 review threshold.

The label is not the outcome

Anaconda, 2,400 CFPB complaints: on the real resolution outcome, 45% zero-shot (below the majority class), 55% at best, behind an XGBoost on word counts; on a rule-defined quick-review question, 82% with the rules written in, while the rules alone scored 94%†. The model answers the question you wrote; calibration is to that question’s labels.

Confidence is a shape, not a truth

`confidence` is peakedness. Guo et al. (2026): a good ranker (AUROC 0.886) whose per-task ECE is 0.168 against 0.074 expected under perfect calibration; OpenRouter: Jev “overestimated its own accuracy mid-range” on Banking77; one pilot found zero mass on the true label in 16% of emotion cases†. And there is no trace: an audit needs a second system that can explain.

Evaluating a decision model: six questions, six numbers

Accuracy alone is the wrong test for a model whose product is a probability; test the probability and the selector.

Question	Measure	Pitfall
Is it right?	accuracy, macro-F1, agreement with human labels (κ)	report the majority baseline (79% in the agent-call test)
Is the probability honest?	ECE with adaptive bins, Brier, log loss; a reliability diagram per question type	ECE depends on binning (Nixon et al. 2019) and is discontinuous (Błasiok et al. 2022); class-wise ECE for many options
Does it know when it is wrong?	AUROC of confidence against correctness; risk–coverage curve, AURC	calibration and ranking are different properties; a well-ranked miscalibrated model still works behind a threshold
Is it stable?	flips under option shuffle, paraphrase, added context; variance over repeated calls	wording, repeats and injected text all move it (slides 24–25)
What does it cost?	$ per 1,000 decisions; p50 and p95 latency at your concurrency	tail latency under load (one call in twenty at 7 s); alias drift between versions
Does it move the business number?	cost-weighted error at your thresholds (Elkan 2001); net benefit curves	TypedBench: underconfident probabilities made the Bayes action cost 2,300 per 10k against 1,080 for the argmax; recalibrate first

Method: a few hundred labelled cases per question type from your own traffic, held out from threshold tuning; rerun on every model version; keep the raw probability vectors.

What the independent evidence says, five weeks in

Strong and cheap at discrimination, a few points under the best generators, calibration that depends on the task.

Task (size)	Jev	Against	Who
Banking77 intents (3,080; 77 options)	81.0%; $0.11 per 1,000	Claude Opus 5 84.4% at $2.42; conf $\geq$0.99 covers 58% at 96.3%; cascade at 0.90: 84.0% for $0.69	OpenRouter†
IMDb sentiment (25,000)	96.5%; $0.65; 22 min	a fine-tuned ModernBERT “similar”	Raschka†
Prompt injection (662)	96.5%; ECE 0.059 (third-party run)	Qwen2.5-7B logprobs 93.0%, ECE 0.061; “indicative only”	Dibia†
Agent tool-call approval (100 calls)	93% zero-shot; 95% with 9 examples	Claude Sonnet 5 98%; majority 79%; 4 of 100 flip on reversal	Archestra†
Faithfulness (88), summaries (50)	95.5%; Brier 0.043; 171 ms (the 88)	GPT-5.6 Luna 94.3%, 0.054, 1,662 ms; summaries’ open rubric: Spearman 0.47 vs 0.72	OpenRouter†
Alignment failures (44 benchmarks)	AUROC 0.886 median (31 yes/no sets); 63$\times$ cheaper than judges	per-benchmark ECE median 0.168 vs 0.074 under perfect calibration; pooled 0.047	Guo et al. 2026
Crash narratives (195,857 $\times$ 27 q.)	F1 0.908 vs 2,416 human labels	raw probabilities overstate prevalence; recalibration cuts the error 3.3$\times$	Rafe, Das 2026
Support tickets, TypedBench (500)	median 0.874	0.708–0.946 across wordings; underconfident	arXiv 2610.11392
CFPB complaints (2,400)	outcome: 45% zero-shot, 55% best; quick review: 82% with rules	XGBoost on word counts leads on the outcome; rules alone 94% on quick review	Anaconda†

Pattern: discrimination close to frontier generators at a few percent of the cost and a tenth of the latency; ECE from 0.02 to 0.25 by task; injected text moves it. Slide 26 is the contract.

One decision, as it goes over the wire

A coding agent asks whether to run a shell command: two Nouls, a Score, a deny-list in front, and thresholds in code.

```
{ "model": "jev-1.13.0",
  "state": {
    "task": "make the unit tests pass",
    "cwd": "/repo", "tool": "bash",
    "command": "rm -rf build/ && npm test" },
  "questions": {
    "reversible": {"type": "noul", "instructions":
      "Can ‘command‘ be undone without data loss?"},
    "serves_task": {"type": "noul", "instructions":
      "Does ‘command‘ plausibly advance ‘task‘?"},
    "blast_radius": {"type": "score", "instructions":
      "Worst case if ‘command‘ is wrong?",
      "criteria": ["none", "local files", "repo-wide",
                   "outside the repo"]} } }

-> "reversible": {"probability": 0.93},
   "serves_task": {"probability": 0.95},
   "blast_radius": {"score": 1.1, "confidence": 0.8}
   usage: 290 input tokens, about $0.00001 (derived)
```

The code around it

`if denylist.matches(cmd): block()` comes *first*: the static list, not the threshold, is the security boundary (OpenRouter cookbook†).

```
if rev>=.9 and ok>=.9 and blast<=1.5: run()
elif rev<.5 or blast>=2.5: ask_human()
else: confirm_with_user()
on timeout(8 s): ask_human()  (fail closed)
```

Log the distributions with the outcome (slides 16, 26).

Where this pattern already runs†

OpenRouter’s hooks for Claude Code, Codex, OpenCode and Cursor; Vercel’s `approval: auto()`; LangChain’s `AutoModeMiddleware`; OpenClaw’s `decisions.evaluate`. All early, all one shape: code decides, the model scores.

Writing questions that work

What the docs advise, what the independent tests add, and what to log so you can prove it later.

Do

Decompose. One judgment per question. A phishing test: one question 62.6%; five sub-questions combined by a logistic regression fitted on 1,000 labels 95.0% (a two-line regex: 91.8%)[†].

Write the rules in. Anaconda's quick-review question reached 82% with the bank's rules in the question, and the rules alone 94%: rules in code, judgment in the question[†].

Describe options with `what`, `not_for` and `examples`; levels with signals; both sides of a Noul.

Batch every question about a state into one call, the speculative ones too; do the arithmetic in code first.

Don't

Don't send the whole history when the question is about the last message: context rot (slide 24).

Don't trust option order. Shuffle, or rotate and average; at minimum measure your flip rate.

Don't threshold on the label. Keep the full probability vector; threshold (slide 18) and recalibrate (slide 16) on it.

Don't ship `jev-latest`. Pin a version; rerun the slide-26 table when you move.

Don't let the state carry instructions. It is untrusted; deny-lists and schema checks go in front (slide 25).

Log the request id, model version, full `probabilities` per question, the threshold that fired and the outcome: without that log there is no reliability diagram, no recalibration, nothing for an auditor.

Build your own: three recipes, in increasing effort

A readout with a temperature, a head on a frozen model, or a fine-tuned encoder; the vendor's number is the floor.

1. Readout + temperature

An afternoon. Serve any open model in vLLM or llama.cpp. Letter the options, prefill "Best answer: [" and read the option logprobs in one prefill, no decode (Li and Wagle 2026: a 4B model, 81.4% on JevBench, ECE 0.057, training-free). Rotate the options and average to kill position bias (AnyJev). Fit one temperature on 300–1,000 labelled cases (Dibia: ECE 0.10 $\rightarrow$ 0.03[†]). Cost: one prefill per state; latency tens of ms on a GPU.

2. A head on a frozen LLM

A week. Freeze a 4–27B base, add LoRA and a pointer or schema head that scores each option at a marker token; train with cross-entropy plus Brier, with ordinal partial credit for Score levels, on public datasets reshaped into typed requests (Kev-4B: 33.8M trainable parameters; Clef: CE + Brier + ordinal term). Serve on llama.cpp `/v1/systemone` at 12–43 ms per question[†]. You get batching, independence between questions and a model you own.

3. A fine-tuned encoder

When you have labels. ModernBERT or DeBERTa with a classification head, cross-entropy, then temperature scaling: Raschka's ModernBERT came within about a point of Jev on IMDb; Red Hat's 200M DeBERTa beat it on prompt injection, 89.0% to 86.4%, at 54 ms on a laptop CPU[†]. Few labels: SetFit (Tunstall et al. 2022). No labels: distil from a judge (Pangakis and Wolken 2024: GPT-4 labels train classifiers as well as human labels on 14 tasks). Fixed label set; no new questions without retraining.

Whichever you build, run the slide-26 table against the vendor's model on the same held-out set. So far (slide 27) a tuned 7B readout or a fine-tuned 200M encoder lands within a few points on narrow tasks, and the hosted model wins on breadth and on zero-shot new questions. Breadth is what the $0.042 per million tokens buys.

Summary: what to take from this lecture

Calibrated classification as a service: the science is old, the economics are new, the evidence is young.

Eight things to remember

1. The output is a distribution over your options; the probability is the product.
2. Confidence summarizes its shape; it is no guarantee.
3. Calibration is a population property scored by proper rules; preference tuning destroys it.
4. Thresholds come from the cost of each mistake, in code.
5. Selective prediction and conformal sets turn a probability into act, confirm or escalate.
6. Cascades put the cheap calibrated model in front of the expensive one.
7. Recalibrate and red-team on your own data; the state is untrusted.
8. The same thing can be built open: readout, head or encoder.

Open questions, October 2026

Is RLCD more than cross-entropy plus Brier? No paper; the open reimplementations get close on narrow tasks and fall short on breadth.

Can calibration survive injected context? 12–73% flips say not yet; the honest fix may be in the harness, not the model.

What is the right benchmark? Accuracy leaderboards reward guessing; the community indices disagree with each other and with the vendor.

Who owns the decision? Every vendor says “code decides”; thresholds, deny-lists and logs are where that promise is kept or broken.

The lecture notes expand every slide, with exercises and a paper kit of about 90 sources.

Sources

Documentation and model cards, papers, and the independent tests quoted; † marks vendor and community material.

Vendor and platform documentation †

TypeSafe docs: system-one, state, primitives (choice, score, noul, box), confidence, models, api/system-one, api/errors, machine-learning-primer, model-jaggedness/jev-1.13, patterns (fan-out, confidence-routing, composite-scoring, intent-routing), cookbooks (parallel_questions, sde_cascade, llm_guardrails, hierarchical_classification, consistency), introduction/coding-agents. docs.typesafe.ai, read Oct 21, 2026.

OpenRouter: Decisions API guide and skill; rankings/decisions; blog: jev-vs-claude-opus-5-classification, jev-vs-llm-as-a-judge, what-is-jev; cookbooks: auto-approve permission prompts, jev-verified cascade; Jev Lab. openrouter.ai.

OpenAI Decisions API guide (developers.openai.com); Microsoft-Decision-1 (commandline.microsoft.com); Cloudflare Clef (blog.cloudflare.com; huggingface.co/Cloudflare/clef); Perplexity pplx-decider-v1.2, jaredpalmer/kev-4b, convaiinnovations/laya, openjev/openjev, SupersonicLabs/Julia-1, togethercomputer/Tev1-4B-experimental (Hugging Face model cards); Liquid AI d1-ultra blog; llama.cpp decision-model support (huggingface.co/blog/ggml-org); Artificial Analysis Decision Index; Jev Decision Index and S1MB (Hugging Face Spaces and blog).

Independent tests †

Anaconda, "Decision time"; Raschka, "Classifier history and Jev"; Dibia, "How Jev works"; Archestra, "We tested Jev on 100 real agent calls"; AbdelStark jev-benchmarks; RINNECODER jev-behavior-study; alexmolas, "Jev can't be calibrated"; Latent Space interview with D. Almeida; TechCrunch (Sept 18, Oct 9), Forbes (Sept 15), Business Wire release (Sept 15).

Papers

Calibration: Brier 1950; DeGroot & Fienberg 1983; Platt 1999; Zadrozny & Elkan 2002; Niculescu-Mizil & Caruana 2005; Gneiting & Raftery 2007; Guo et al. 2017; Nixon et al. 2019; Kumar, Liang & Ma 2019; Kull et al. 2019; Ovadia et al. 2019; Minderer et al. 2021; Błasiok et al. 2022. LLM calibration: Zhao et al. 2021; Lin, Hilton & Evans 2022; Kadavath et al. 2022; OpenAI 2023; Tian et al. 2023; Xiong et al. 2023; Zhu et al. 2023; Kapoor et al. 2024; Xiao et al. 2025; Sanz-Guerrero & von der Wense 2026. RL for calibration: Band et al. 2024; Stengel-Eskin et al. 2024 (LACIE); Xu et al. 2024 (SaySelf); Leng et al. 2024; Bani-Harouni et al. 2025; Damani et al. 2025 (RLCR); Mei et al. 2025; Zeng et al. 2025; Kalai et al. 2025; Yang et al. 2023 (RLCD, contrastive). Selection: Chow 1970; El-Yaniv & Wiener 2010; Geifman & El-Yaniv 2017; Geifman et al. 2019; Madras et al. 2018; Mozannar & Sontag 2020; Hendrycks & Gimpel 2016; Elkan 2001; Vickers & Elkin 2006; Kirichenko et al. 2025. Conformal: Vovk et al. 2005; Angelopoulos & Bates 2021; Kumar et al. 2023; Quach et al. 2023; Ren et al. 2023; Mohri & Hashimoto 2024. Classifiers and judges: Hendrycks et al. 2020; Robinson et al. 2022; Zheng et al. 2023 (selectors); Pezeshkpour & Hruschka 2023; Wang et al. 2024; Wang et al. 2023; Liu et al. 2023 (G-Eval); Zheng et al. 2023 (judge); Kim et al. 2024; Lambert et al. 2024; Inan et al. 2023; Tunstall et al. 2022; Warner et al. 2024; Pangakis & Wolken 2024; Hinton et al. 2015; Hsieh et al. 2023. Cascades: Chen et al. 2023; Jitkrittum et al. 2023; Yue et al. 2023; Ong et al. 2024; Cheng et al. 2023. System 1/2: Kahneman 2011; Booch et al. 2020; Yu et al. 2024. Decision models: Rafe & Das 2026; Guo et al. 2026; Hu et al. 2026; Xu et al. 2026; Wu & Lim 2026; Li & Wagle 2026; Sharma et al. 2026 (TypedBench). Full list with arXiv identifiers in the paper kit.